

我是 TOC

GYF

2023 年 12 月

计算理论基础复习提要

李元《计算理论基础》复习提要。

制作者：GYF

GitHub: <https://github.com/23300240002>

第一节课复习资料

一、本节课知识点讲解

1.1 渐近记号 (Asymptotic Notation)

渐近记号用于描述函数在自变量趋于无穷时的增长趋势，是分析算法时间复杂度的核心工具。

大 O 记号 (Big-O) 定义: $f(n) = O(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长速度不超过 $g(n)$ 的常数倍, 描述上界。

例子: $3n^2 + 2n + 1 = O(n^2)$ 。

Ω 记号 (Omega) 定义: $f(n) = \Omega(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: 描述下界。

Θ 记号 (Theta) 定义: $f(n) = \Theta(g(n))$ 表示同时有 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 。

含义: $f(n)$ 与 $g(n)$ 同阶增长, 描述精确增长率。

小 o 记号 (Little-o) 定义: $f(n) = o(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长严格慢于 $g(n)$ 。

例子: $2n = o(n^2)$ 。

ω 记号 (Little-omega) 定义: $f(n) = \omega(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: $f(n)$ 的增长严格快于 $g(n)$ 。

常见增长率排序 (从慢到快) 常数 $O(1) <$ 对数 $O(\log n) <$ 线性 $O(n) <$ 线性对数 $O(n \log n) <$ 多项式 $O(n^c)$ ($c > 1$) $<$ 指数 $O(c^n)$ ($c > 1$) $<$ 阶乘 $O(n!)$ 等。

注意: 比较增长率时需谨慎, 如 $2^{\log^2 n}$ 虽为指数形式, 但因指数是 $\log^2 n$, 故实际增长比多项式快但比真指数 (如 e^n) 慢。

1.2 字母表和语言 (Alphabets and Languages)

基本概念

- 字母表 Σ : 有限符号的集合, 如 $\Sigma = \{0, 1\}$ 。
- 字符串: 字母表中符号的无穷序列, 空串记为 ε 。
- 克林闭包 Σ^* : 由 Σ 中符号构成的所有字符串 (含空串) 的集合。
- 语言: Σ^* 的子集, 即一些字符串的集合。

语言运算

- 并、交、补: 与集合运算相同。
- 连接: $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$ 。
- 克林星号: $L^* = \{w_1 w_2 \dots w_k \mid k \geq 0, w_i \in L\}$, 即 L 中字符串的任意有限次连接。 $L^+ = L L^*$ 表示至少一次连接。(这是因为克林星号允许空串, 这里的 L^+ 正闭包强制一个因子来自 L , 不允许空串存在)

1.3 问题编码 (Encoding of Problems)

在计算理论中, 将问题实例编码为字符串以便计算模型 (如图灵机) 处理。编码应合理、简洁, 且能在多项式时间内解码。

二、作业题目详细讲解

题目 1

题目

证明

$$\frac{1}{2 \ln 2} + \frac{1}{3 \ln 3} + \dots + \frac{1}{n \ln n} = \ln \ln n + O(1)$$

和

$$\frac{1}{1^{1.1}} + \frac{1}{2^{1.1}} + \dots + \frac{1}{n^{1.1}} = O(1).$$

讲解

第一个等式证明 估计级数 $\sum_{k=2}^n \frac{1}{k \ln k}$ 。考虑函数 $f(x) = \frac{1}{x \ln x}$ ，在 $[2, \infty)$ 上为正且单调递减。对单调递减正函数，有

$$f(k+1) \leq \int_k^{k+1} f(x) dx \leq f(k) \quad (k \geq 2).$$

从 $k=2$ 累加到 n ，并将 $f(2)$ 单独留在上界中，得到

$$\int_2^{n+1} f(x) dx \leq \sum_{k=2}^n f(k) \leq f(2) + \int_2^n f(x) dx.$$

两侧都与 $\int_2^n f(x) dx$ 相比仅差有限常数，因此

$$\sum_{k=2}^n f(k) = \int_2^n f(x) dx + O(1).$$

计算积分：

$$\int \frac{1}{x \ln x} dx = \int \frac{1}{\ln x} d(\ln x) = \ln |\ln x| + C.$$

所以

$$\int_2^n \frac{1}{x \ln x} dx = \ln \ln n - \ln \ln 2.$$

因此

$$\sum_{k=2}^n \frac{1}{k \ln k} = \ln \ln n - \ln \ln 2 + O(1) = \ln \ln n + O(1),$$

其中常数 $-\ln \ln 2$ 被吸收到 $O(1)$ 中。

第二个等式证明 考虑级数 $\sum_{k=1}^n \frac{1}{k^{1.1}}$ 。由于指数 $1.1 > 1$ ，这是 p -级数且 $p > 1$ ，故无穷级数 $\sum_{k=1}^{\infty} \frac{1}{k^{1.1}}$ 收敛。收敛意味着部分和有上界，即存在常数 M 使得对所有 n 有 $\sum_{k=1}^n \frac{1}{k^{1.1}} \leq M$ ，所以 $\sum_{k=1}^n \frac{1}{k^{1.1}} = O(1)$ 。

更具体地，也可用积分估计上界：

$$\sum_{k=1}^n \frac{1}{k^{1.1}} \leq 1 + \int_1^n \frac{1}{x^{1.1}} dx = 1 + \left[\frac{x^{-0.1}}{-0.1} \right]_1^n = 1 + 10(1 - n^{-0.1}) \leq 11.$$

故确有界。

总结：第一个级数发散，发散速度为 $\ln \ln n$ ；第二个级数收敛，部分和有界。

题目 2

题目

将以下函数按渐近增长率排序（若 $f(n) = O(g(n))$ ，则 f 排在 g 前面）： $n^n - \sqrt{n} - \log \log n - 0.001n^7 - 5000n - e^n - 2^{\log^2 n} - 10^{10^{10}} - n! - 2^{2^n}$

讲解

需比较函数增长率，按从慢到快排序。分析各函数：

1. $10^{10^{10}}$ ：常数，增长率最低。
2. $\log \log n$ ：双对数，增长极慢，但比常数快。
3. $\sqrt{n} = n^{0.5}$ ：多项式（指数 0.5），比对数函数快（任何 n^c ($c > 0$) 均比 $\log n$ 快，而 $\log \log n$ 更慢）。
4. $5000n$ ：线性函数 n^1 ，比 $n^{0.5}$ 快。
5. $0.001n^7$ ：7 次多项式，比线性快。
6. $2^{\log^2 n}$ ：注意 $f(n) = 2^{\log^2 n}$ ，取对数得 $\log f(n) = \log^2 n$ 。多项式 n^c 的对数为 $c \log n$ ， $\log^2 n$ 增长比 $c \log n$ 快，故 $2^{\log^2 n}$ 比任何多项式快。但与指数函数 e^n 比较： $\log(e^n) = n$ ， n 增长比 $\log^2 n$ 快得多，所以 $2^{\log^2 n}$ 比 e^n 慢。因此它介于多项式与指数之间。
7. e^n ：指数函数，增长快。
8. $n!$ ：阶乘。由斯特林公式 $n! \sim \sqrt{2\pi n}(n/e)^n$ ，取对数 $\log(n!) \sim n \log n - n$ ，而 $\log(e^n) = n$ ， $n \log n$ 增长快于 n ，故 $n!$ 比 e^n 快。
9. n^n ：增长很快。
10. 2^{2^n} ：双指数函数，取对数得 $\log(2^{2^n}) = 2^n$ ，而 $\log(n^n) = n \log n$ ， 2^n 远快于 $n \log n$ ，故 2^{2^n} 比 n^n 快。

排序（从慢到快）：

$$10^{10^{10}} \prec \log \log n \prec \sqrt{n} \prec 5000n \prec 0.001n^7 \prec 2^{\log^2 n} \prec e^n \prec n! \prec n^n \prec 2^{2^n}$$

其中 $\prec$ 表示增长率小于。

关键点验证： $2^{\log^2 n}$ ：比多项式快（因为 $\log^2 n$ 比 $c \log n$ 快），但比指数慢（因为 $\log^2 n$ 比 n 慢）。 $n!$ 与 e^n ： $\log(n!) \sim n \log n > n = \log(e^n)$ ，故 $n!$ 更快。 n^n 与 2^{2^n} ： $\log(n^n) = n \log n$ ， $\log(2^{2^n}) = 2^n$ ， 2^n 更快。

题目 3

题目

令 $\Sigma = \{0, 1\}$ ，定义语言

$$L = \{w \in \{0,1\}^* : w \text{ 中 } 0 \text{ 和 } 1 \text{ 的数量不相等}\}.$$

证明 $L^* = \Sigma^*$ 。

讲解

需证集合相等： $L^* \subseteq \Sigma^*$ 且 $\Sigma^* \subseteq L^*$ 。

1. $L^* \subseteq \Sigma^*$ ：显然，因 L 中字符串均由 0 和 1 组成，故 L^* 中字符串也由 0 和 1 组成，属于 Σ^* 。
2. $\Sigma^* \subseteq L^*$ ：需证任意 $w \in \Sigma^*$ 可表示为 L 中若干字符串的连接。分情况：
 - 若 w 中 0 和 1 数量不相等，则 $w \in L \subseteq L^*$ 。
 - 若 w 中 0 和 1 数量相等，取最短前缀 x 使 x 中 0 和 1 数量差为 1，则剩余部分 y 的差为 -1 ，故 $x, y \in L$ ，从而 $w = xy \in L^*$ 。

综上， $\Sigma^* \subseteq L^*$ ，故 $L^* = \Sigma^*$ 。

复习小结：第一节课重点掌握渐近记号的含义与比较、积分估计级数和方法、语言运算及克林闭包的概念，并能进行简单的集合相等证明。

第二节课复习资料

一、本节课知识点讲解

2.1 有限自动机的定义

有限自动机 (FA) 是一种最简单的计算模型，它读取输入字符串，根据当前状态和输入符号决定下一个状态，最终根据结束状态是否在接受状态集合中来决定是否接受该字符串。

形式化定义（五元组） 一个有限自动机 M 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ：

1. Q ：有限状态集合。
2. Σ ：输入字母表（有限符号集合）。
3. $\delta : Q \times \Sigma \rightarrow Q$ ：转移函数。给定当前状态和输入符号，输出下一个状态。
4. $q_0 \in Q$ ：初始状态。
5. $F \subseteq Q$ ：接受状态集合。

FA 如何接受字符串 设 $w = w_1 w_2 \cdots w_n$ ，其中每个 $w_i \in \Sigma$ 。 M 从 q_0 开始，依次读入每个符号，根据 δ 转移状态。读完所有符号后，若最终状态属于 F ，则 M 接受 w ；否则拒绝。

形式化： M 接受 w 当存在状态序列 $r_0, r_1, \dots, r_n$ 满足：

- $r_0 = q_0$
- $\delta(r_i, w_{i+1}) = r_{i+1}$ (对 $i = 0, \dots, n-1$)
- $r_n \in F$

正则语言 如果一个语言 L 被某个有限自动机接受，则称 L 是正则语言。记 $L(M)$ 为 M 接受的所有字符串的集合，即 M 识别的语言。

2.2 设计有限自动机的思路

设计 FA 的关键在于确定状态的含义。通常，状态用来记录读入部分字符串后所处的“情形”。例如：

- 记录当前读入的二进制数模某个数的余数。
- 记录是否已经出现了某个禁止子串。
- 记录最近读入的几个字符，以判断是否满足模式。

2.3 正则语言的封闭性

正则语言在多种运算下是封闭的，即对正则语言进行这些运算得到的语言仍然是正则的。封闭性证明通常通过构造新的 FA 来实现。

以下是常见的封闭运算（设 A, B 是正则语言）：

1. 补运算： $\overline{A} = \{w \in \Sigma^* \mid w \notin A\}$ 。
 - 证明：设 M 识别 A ，则交换 M 的接受与非接受状态得到 M' ， M' 识别 $\overline{A}$ 。
2. 并运算： $A \cup B$ 。
 - 证明：使用乘积自动机。设 M_1 识别 A ， M_2 识别 B ，构造 M 的状态为 M_1 和 M_2 状态的笛卡尔积（也就是一个状态变为“二元”的），转移模拟两者并行运行，接受状态定义为只要有一个自动机接受即可。
3. 交运算： $A \cap B$ 。
 - 证明：类似并运算，但接受状态定义为两个自动机同时接受。
4. 连接运算： $A \circ B = \{xy \mid x \in A, y \in B\}$ 。
 - 证明思路：非确定性有限自动机（NFA）更容易构造，参见第三讲知识点。
5. 星号运算： $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0, \text{每个 } x_i \in A\}$ 。
 - 证明思路：非确定性有限自动机（NFA）更容易构造，参见第三讲知识点。

注意：第二课中先介绍的是确定性有限自动机（DFA），但封闭性证明有时需要非确定性有限自动机（NFA）的概念，可见第三课。

二、作业题目详细讲解

题目 1

题目

设计一个有限自动机，接受以下语言

$$L = \left\{ w \in \{0, 1\}^* \mid w = w_1 w_2 \cdots w_n \text{ 使得 } \sum_{i=1}^n w_i 2^{n-i} \equiv 0 \pmod{5} \right\}.$$

换句话说，语言 L 是所有表示 5 的倍数的二进制字符串的集合，字符串以最高有效位优先（MSB）书写。绘制该有限自动机的状态图。

讲解

这个语言 L 中的字符串，如果解释为一个二进制数（最高位在前），这个数能被 5 整除。我们需要设计一个 DFA 来识别这样的字符串。

设计思路：

DFA 的状态应该记录当前已经读入的部分二进制数模 5 的余数。因为每读入一位，新的数值等于原来的数值左移一位（乘以 2）再加上新读入的位（0 或 1），然后模 5。所以，如果我们用状态 q_i 表示当前余数为 i ($i = 0, 1, 2, 3, 4$)，那么转移函数可以这样定义：

从状态 q_i 读入符号 b (0 或 1)，转移到状态 q_j ，其中 $j = (2 \times i + b) \bmod 5$ 。

初始状态应该是 q_0 ，因为还没有读入任何数字时，数值为 0，余数为 0。

接受状态也是 q_0 ，因为整个字符串读完时，我们希望余数为 0。

具体转移计算：

我们列出所有可能的状态和输入组合：

- 从 q_0 读入 0：新余数 $= (2 \times 0 + 0) \bmod 5 = 0$ ，所以转移到 q_0 。
- 从 q_0 读入 1：新余数 $= (2 \times 0 + 1) \bmod 5 = 1$ ，转移到 q_1 。
- 从 q_1 读入 0：新余数 $= (2 \times 1 + 0) \bmod 5 = 2$ ，转移到 q_2 。
- 从 q_1 读入 1：新余数 $= (2 \times 1 + 1) \bmod 5 = 3$ ，转移到 q_3 。
- 从 q_2 读入 0：新余数 $= (2 \times 2 + 0) \bmod 5 = 4$ ，转移到 q_4 。
- 从 q_2 读入 1：新余数 $= (2 \times 2 + 1) \bmod 5 = 5 \equiv 0$ ，转移到 q_0 。
- 从 q_3 读入 0：新余数 $= (2 \times 3 + 0) \bmod 5 = 6 \equiv 1$ ，转移到 q_1 。
- 从 q_3 读入 1：新余数 $= (2 \times 3 + 1) \bmod 5 = 7 \equiv 2$ ，转移到 q_2 。
- 从 q_4 读入 0：新余数 $= (2 \times 4 + 0) \bmod 5 = 8 \equiv 3$ ，转移到 q_3 。
- 从 q_4 读入 1：新余数 $= (2 \times 4 + 1) \bmod 5 = 9 \equiv 4$ ，转移到 q_4 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2, q_3, q_4\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： q_0
- 转移函数如上表。

我们可以绘制状态图：五个状态围成一个圆圈（顺序可为 q_0, q_1, q_2, q_3, q_4 ），每个状态根据上述转移有两条出边。注意 q_0 是初始状态（通常用箭头指向）和接受状态（通常用双圈表示）。

重要提醒：这个 DFA 也会接受空串 ε ，因为初始状态就是接受状态。在二进制数的表示中，空串通常不被视为合法的数字表示，但题目没有特别说明，按照题目定义，空串对应的数值为 0， $0 \bmod 5 = 0$ ，所以应该接受。如果要求非空字符串，则需要稍作修改，但题目没有此要求，所以这样设计是合理的。

题目 2

题目

设计一个有限自动机，识别以下语言

$$L = \{w \in \{0, 1\}^* : w \text{ 不包含子串 } 10\}.$$

绘制其状态图。

讲解

语言 L 包含所有不出现子串“10”的二进制字符串。也就是说，字符串中一旦出现了 1，后面就不能再出现 0，否则就会形成“10”。因此，合法的字符串必须是这样的形式：先有零个或多个 0，然后有零个或多个 1，即所有 0 都在所有 1 的前面。用正则表达式表示就是 0^*1^* 。

设计思路：

我们可以用三个状态来设计 DFA：

- 状态 q_0 ：表示目前还处于“0 的阶段”，即到目前为止读入的部分全部是 0（或者还没有读入任何字符）。在这个状态，我们还没有遇到 1。
- 状态 q_1 ：表示我们已经读到了至少一个 1，并且还没有出现 0 跟在 1 后面的情况。在这个状态，我们只希望后面继续读入 1，如果读到 0，就违反规则。
- 状态 q_2 ：表示已经出现了禁止子串“10”，即进入了“死亡状态”，一旦进入这个状态，无论读入什么符号，都将永远停留在这个状态，并且这个状态不是接受状态。

初始状态是 q_0 ，因为开始时还没有读入任何字符，属于 0 的阶段。

接受状态应该是 q_0 和 q_1 ，因为在这两个状态下，都没有出现禁止子串。

转移函数：

- 从 q_0 读入 0：仍然处于 0 的阶段，保持 q_0 。
- 从 q_0 读入 1：进入了 1 的阶段，转移到 q_1 。
- 从 q_1 读入 1：仍然在 1 的阶段，保持 q_1 。
- 从 q_1 读入 0：形成了“10”，禁止，转移到死亡状态 q_2 。
- 从 q_2 读入任何符号（0 或 1）：已经违规，永远留在 q_2 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： $\{q_0, q_1\}$
- 转移函数：
 - $q_0 \xrightarrow{0} q_0$
 - $q_0 \xrightarrow{1} q_1$
 - $q_1 \xrightarrow{0} q_2$
 - $q_1 \xrightarrow{1} q_1$

$$\begin{aligned} & - q_2 \xrightarrow{0} q_2 \\ & - q_2 \xrightarrow{1} q_2 \end{aligned}$$

注意: q_2 不是接受状态, 且所有从 q_2 出发的转移都指向自己。

注意: 这个 DFA 也接受空串, 因为初始状态 q_0 是接受状态。语言 0^*1^* 确实包含空串。

题目 3

题目

使用大 O 符号的定义证明以下内容:

- 1) 如果 $f_1 = O(g_1)$ 且 $f_2 = O(g_2)$, 则 $f_1 f_2 = O(g_1 g_2)$ 。
- 2) 如果 $f_1 = O(g)$ 且 $f_2 = O(g)$, 则 $f_1 + f_2 = O(g)$ 。

讲解

我们需要回顾大 O 符号的严格定义:

$f(n) = O(g(n))$ 意味着存在正常数 C 和 N , 使得对于所有 $n > N$, 都有 $|f(n)| \leq C|g(n)|$ 。

证明第 1 小题 已知: $f_1 = O(g_1)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g_1(n)|$ 。

已知: $f_2 = O(g_2)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g_2(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

将两个不等式相乘:

$$|f_1(n)| \cdot |f_2(n)| \leq C_1|g_1(n)| \cdot C_2|g_2(n)| = (C_1 C_2)|g_1(n)g_2(n)|。$$

由于 $|f_1(n)f_2(n)| = |f_1(n)| \cdot |f_2(n)|$, 故有

$$|f_1(n)f_2(n)| \leq (C_1 C_2)|g_1(n)g_2(n)|。$$

因此, 取 $C = C_1 C_2$, 则对于所有 $n > N$, 不等式成立。所以 $f_1 f_2 = O(g_1 g_2)$ 。

证明第 2 小题 已知: $f_1 = O(g)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g(n)|$ 。

已知: $f_2 = O(g)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g(n)|$ 。

目标: 证明 $f_1 + f_2 = O(g)$, 即要找到常数 C 和 N , 使得对所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

利用三角不等式:

$$|f_1(n) + f_2(n)| \leq |f_1(n)| + |f_2(n)| \leq C_1|g(n)| + C_2|g(n)| = (C_1 + C_2)|g(n)|。$$

因此, 取 $C = C_1 + C_2$, 则对于所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。所以 $f_1 + f_2 = O(g)$ 。

复习小结: 第二节课重点掌握有限自动机的形式化定义、设计方法 (特别是基于余数或禁止模式的状态设计), 以及正则语言封闭性的基本证明思路 (补、并)。同时, 大 O 记号的性质证明需严格按照定义进行构造。

第三节课复习资料

一、本节课知识点讲解

3.1 非确定性有限自动机 (NFA)

NFA 的定义 NFA 与 DFA 类似，但转移函数可以是非确定性的：即从一个状态读入一个符号，可以转移到多个可能的状态（包括零个）。此外，NFA 允许 ε 转移，即不读入任何符号就改变状态。

形式化定义：一个 NFA 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ，其中：

- Q ：有限状态集合。
- Σ ：输入字母表。
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$ ，其中 $\mathcal{P}(Q)$ 是 Q 的幂集（即 Q 中所有状态都可能存在， 2^Q 种）。
- $q_0 \in Q$ ：初始状态。
- $F \subseteq Q$ ：接受状态集合。

NFA 接受一个字符串 w ，如果存在一条从初始状态到某个接受状态的路径，路径上的转移标记依次与 w 的符号匹配（允许 ε 转移，不消耗输入符号）。

NFA 与 DFA 的等价性 对于任意 NFA，存在一个 DFA 接受相同的语言（反之亦然）。DFA 本身显然就是一种 NFA，而反过来证明可使用子集构造法：将 NFA 的状态集合的子集作为 DFA 的状态（关键！我们构造的 DFA，其状态应是已知 NFA 的某些状态的集合），以此模拟 NFA 所有可能的并行状态。

子集构造法：给定 NFA $N = (Q, \Sigma, \delta, q_0, F)$ ，构造 DFA $M = (Q', \Sigma, \delta', q'_0, F')$ ：

- $Q' = \mathcal{P}(Q)$ 。
- $q'_0 = E(\{q_0\})$ ，其中 $E(R)$ 表示从集合 R 中的状态通过 ε 转移所能到达的状态集合（ ε 闭包），相当于把 NFA 中可以用来出发的状态“打包”了。
- 对于 $S \subseteq Q$ 和 $a \in \Sigma$ ，定义 $\delta'(S, a) = E\left(\bigcup_{r \in S} \delta(r, a)\right)$ 。这样，我们就能得到我们构造的 DFA 中的一个“打包”遇到一个输入字母时，应去往哪一个“打包”。
- $F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$ 。因为我们的最终目标是让构造的 DFA 和原始的 NFA 等价，因此凡是包含 F 中状态的的状态集合都应作为我们构造的 DFA 的接收状态。

正则语言的封闭性（连接与星号） 利用 NFA 可以方便地证明正则语言在连接和星号运算下封闭。

- 连接：若 L_1 和 L_2 是正则语言，则 $L_1 \circ L_2 = \{xy : x \in L_1, y \in L_2\}$ 也是正则的。证明：设 N_1 接受 L_1 ， N_2 接受 L_2 。构造 NFA N ：将 N_1 的每个接受状态通过 ε 转移连接到 N_2 的初始状态。 N 的初始状态为 N_1 的初始状态，接受状态为 N_2 的接受状态。
- 星号：若 L 是正则语言，则 $L^* = \{x_1 x_2 \cdots x_k : k \geq 0, x_i \in L\}$ 也是正则的。证明：设 N_1 接受 L 。构造 NFA N ：新增一个初始状态 q_0 （也是接受状态），从 q_0 通过 ε 转移到 N_1 的初始状态；同时从 N_1 的接受状态通过 ε 转移回到 N_1 的初始状态（这样就允许了重复）。

3.2 正则表达式

正则表达式是一种描述正则语言的形式化方法。递归定义如下：

1. 基础:

- a ($a \in \Sigma$) 是正则表达式, 表示语言 $\{a\}$ 。
- ε 是正则表达式, 表示语言 $\{\varepsilon\}$ 。(注意, 这里的 $\{\varepsilon\}$ 中只包含一个元素, 即空串, 不能和空集混淆)
- $\emptyset$ 是正则表达式, 表示空语言。

2. 归纳:

- 如果 r 和 s 是正则表达式, 表示语言 $L(r)$ 和 $L(s)$, 则:
 - $(r \cup s)$ 是正则表达式, 表示 $L(r) \cup L(s)$ 。
 - $(r \circ s)$ 是正则表达式, 表示 $L(r) \circ L(s)$ 。
 - (r^*) 是正则表达式, 表示 $L(r)^*$ 。

通常, 运算符优先级为: 星号 > 连接 > 并集。括号可省略。

正则表达式与有限自动机等价: 每个正则表达式对应一个正则语言, 反之亦然。证明可通过将正则表达式转换为 NFA (递归构造), 或反之 (状态消去法)。

二、作业题目详细讲解

题目 1

题目

定义一个反向读取 DFA 为从右向左处理输入字符串的确定性有限自动机。证明一个语言是正则的当且仅当它可以被某个反向读取 DFA 接受。

讲解

我们需要证明两个方向。

方向 1: 如果语言 L 是正则的, 那么存在一个反向读取 DFA 接受 L 。

证明: 因为 L 是正则语言, 所以存在一个普通的 DFA $M = (Q, \Sigma, \delta, q_0, F)$ 接受 L 。我们构造一个反向读取 DFA M' 如下:

- M' 的状态集与 M 相同: $Q' = Q$ 。
- M' 的输入字母表与 M 相同: $\Sigma' = \Sigma$ 。
- M' 的转移函数 δ' 定义为: 对于任意 $q \in Q$ 和 $a \in \Sigma$, 令 $\delta'(q, a) = p$ 当且仅当在 M 中有 $\delta(p, a) = q$ 。也就是说, M' 的转移是 M 转移的反向。
- M' 的初始状态设为 M 的任意一个接受状态。

上述做法在转移函数构造和初始状态构造上, 是否存在不严谨的地方呢? 实际是有的: 1) 把 $\delta'(q, a)$ 定义为“所有前驱”后, 它可能不是单值, 所得自动机其实是 NFA 而非 DFA; 2) 初始状态不能随便选取一个接受状态, 而应该同时考虑所有接受状态的反向可达性, 否则会漏掉路径。

可以考虑: 先构造一个 NFA N , 使得 N 接受 L 的反转语言 $L^R = \{w^R : w \in L\}$, 其中 w^R 是 w 的翻转。具体构造:

- 先在 M 的基础上新增一个初始状态 s ，从 s 通过 ε 转移到每个 $f \in F$ ；设 $N = (Q \cup \{s\}, \Sigma, \delta_N, s, \{q_0\})$ 。将这个 s 作为初始状态，就巧妙地解决了上述的“所有接受状态的反向可达性”问题，因为 s 可以到达这些状态，现在就等价于 M 的最终状态了。
- 定义反向转移：对任意 $q \in Q, a \in \Sigma$ ，令 $\delta_N(q, a) = \{p \in Q : \delta(p, a) = q\}$ （这样就解决了上述“可能不是单值”问题）；其他未定义的转移为空集。
- 最后， $\{q_0\}$ 可构造为唯一接受状态。

这个 NFA N 接受的语言正好是 L^R 。因为 L 是正则的，所以 L^R 也是正则的（正则语言在反转下封闭，已证结论）。然后，将 NFA N 通过子集构造法转化为 DFA D （前面已证），这个 DFA D 接受 L^R 。最后，说明让其反向读取即可。因此，存在反向读取 DFA 接受 L 。

方向 2：如果存在一个反向读取 DFA 接受语言 L ，那么 L 是正则的。

证明：设 M' 是一个反向读取 DFA，接受 L 。定义 M' 正向读（从左向右）时接受的语言为 L' 。由于 M' 是 DFA，所以 L' 是正则语言。但注意 M' 反向读时接受 L ，正向读时实际上接受的是 L 的反转 L^R 。因此 $L^R = L'$ 是正则的。由于正则语言在反转下封闭，所以 $L = (L^R)^R$ 也是正则的。因此直接得证。

综上，语言 L 是正则的当且仅当它可以被某个反向读取 DFA 接受。

题目 2

题目

给出语言 $L = \{w \in \{0,1\}^* : w \text{ 中 } 1 \text{ 的个数是偶数, 且 } 0 \text{ 的个数也是偶数}\}$ 的正则表达式。

讲解

我们想要所有包含偶数个 0 和偶数个 1 的二进制字符串。考虑用模 2 的状态来思考。但直接写正则表达式，可以注意到：任何这样的字符串，都可以看成由以下四种“片段”组成，这些片段在添加时不会改变 0 和 1 的奇偶性（即添加后，0 和 1 的个数的奇偶性保持不变）：

- “00”：添加两个 0，不改变奇偶性。
- “11”：添加两个 1，不改变奇偶性。
- “01”后跟“10”：即“0110”，整体添加了两个 0 和两个 1，奇偶性不变。类似地，“10”后跟“01”也是。

因此，最简洁的表达式是：

$$(00 \cup 11 \cup (01 \cup 10)(00 \cup 11)^*(01 \cup 10))^*$$

解释：

- $00 \cup 11$ ：添加两个相同字符，奇偶性不变。
- $(01 \cup 10)(00 \cup 11)^*(01 \cup 10)$ ：先添加一个“01”或“10”，这会翻转 0 和 1 的奇偶性（使 0 和 1 都变成奇数个），然后添加任意个“00”或“11”（不改变奇偶性），最后再添加一个“01”或“10”，再次翻转奇偶性，回到偶数。整体效果是奇偶性不变。

整个表达式外面加星号，表示重复任意多次（包括零次），所以最终得到的就是所有偶数个 0 和偶数个 1 的字符串。

注意：这个表达式也包含空串，因为星号允许零次重复，而空串中 0 和 1 的个数都是 0（偶数），符合要求。

题目 3

题目

构造与以下正则表达式对应的 NFA: 1. $0^+ \cup (01)^*$ 2. $(0 \cup 1^+)0^*1^+$

讲解

我们分别构造两个正则表达式的 NFA。NFA 允许 ϵ 转移，并且每个部分可以独立构造然后组合。

第一问: $0^+ \cup (01)^*$ 这个语言表示: 要么是一个或多个 0 的序列, 要么是零个或多个 “01” 的序列。

我们构造 NFA 如下:

- 状态集合: $\{q_0, q_1, q_2, q_3, q_4, q_5\}$, 其中 q_0 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_2 (对于 0^+ 部分) 以及 q_3 和 q_5 (对于 $(01)^*$ 部分)。

转移函数:

1. 从 q_0 通过 ϵ 转移到 q_1 (进入 0^+ 分支) 和通过 ϵ 转移到 q_3 (进入 $(01)^*$ 分支)。
2. 0^+ 部分:
 - $q_1 \xrightarrow{0} q_2$ 。
 - $q_2 \xrightarrow{0} q_2$ (自环, 允许更多 0)。
3. $(01)^*$ 部分:
 - q_3 是接受状态 (因为 $(01)^*$ 允许空串)。
 - $q_3 \xrightarrow{0} q_4$ 。
 - $q_4 \xrightarrow{1} q_5$ 。
 - q_5 是接受状态。
 - $q_5 \xrightarrow{0} q_4$ (以便继续下一个 “01”)。

这样, 对于 $(01)^*$, 空串可以直接在 q_3 接受; 非空的 “01” 序列会经过 $q_3 \rightarrow q_4 \rightarrow q_5$, 并且从 q_5 读 0 可以回到 q_4 继续循环。

第二问: $(0 \cup 1^+)0^*1^+$ 这个语言表示: 首先是一个 0 或者一个或多个 1, 然后是零个或多个 0, 最后是一个或多个 1。

我们构造 NFA 如下:

- 状态集合: $\{q_1, q_2, q_3, q_4, q_5\}$, 其中 q_1 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_5 。

转移函数:

1. 第一部分 $(0 \cup 1^+)$:
 - $q_1 \xrightarrow{0} q_2$ (选择 0)。
 - $q_1 \xrightarrow{1} q_3$ (选择 1^+)。
 - $q_3 \xrightarrow{1} q_3$ (自环, 允许更多 1)。

2. 连接第二部分 0^* :

- 从 q_2 和 q_3 通过 ε 转移到 q_4 。
- $q_4 \xrightarrow{0} q_4$ (自环, 允许任意多个 0)。

3. 连接第三部分 1^+ :

- $q_4 \xrightarrow{1} q_5$ (第一个 1)。
- $q_5 \xrightarrow{1} q_5$ (自环, 允许更多 1)。

这样, NFA 从 q_1 开始, 根据第一个字符选择分支, 然后通过 ε 转移到 q_4 , 在 q_4 可以读任意多个 0, 最后必须读至少一个 1 到达 q_5 并可以继续读更多 1。 q_5 是接受状态。

注意: 这个构造中, 从 q_2 和 q_3 到 q_4 的 ε 转移是必要的, 因为 0^* 部分允许零个 0, 所以不能直接合并状态。而 q_4 到 q_5 的转移是读 1, 没有使用 ε 转移, 这样更简洁。

复习小结: 第三节课重点掌握 NFA 的定义及其与 DFA 的等价性 (子集构造法)、正则表达式的递归定义、以及利用 NFA 证明正则语言在连接和星号运算下的封闭性。同时, 要能够进行正则表达式与有限自动机之间的转换 (构造 NFA)。作业题目涉及反向 DFA 的等价性证明、构造特定语言的正则表达式、以及从正则表达式构造 NFA, 需要理解并熟练运用相关概念和构造技巧。

第四节课复习资料

一、本节课知识点讲解

4.1 泵引理 (Pumping Lemma)

泵引理是用于证明一个语言不是正则的有力工具。其基本思想是: 如果一个语言是正则的, 那么它能被一个 DFA 识别。DFA 只有有限个状态 (设其状态数为 p), 那么任何长度大于 p 的字符串在运行过程中必然会重复经过某个状态。利用这个重复, 我们可以将字符串的某部分 “泵入” 任意多次, 得到的字符串仍然应该被接受。

泵引理 (正则语言的泵引理): 设 L 是一个正则语言, 则存在一个正整数 p (称为泵长度) 使得对于 L 中任意长度至少为 p 的字符串 s , 都可以将 s 写成 $s = xyz$ 的形式, 满足以下三个条件:

1. 对任意 $i \geq 0$, $xy^iz \in L$;
2. $|y| > 0$;
3. $|xy| \leq p$ 。

需要注意的是, 泵引理是正则语言的必要条件, 而不是充分条件。也就是说, 如果一个语言满足泵引理的条件, 我们不能说它一定是正则的; 但如果不满足, 则它一定不是正则的。

证明思路: 设 M 是一个有 p 个状态的 DFA, 且 $L(M) = L$ 。对于任意长度 $n \geq p$ 的字符串 $s = s_1s_2 \cdots s_n \in L$, 考虑 M 在输入 s 上的状态序列 $r_0, r_1, \dots, r_n$, 其中 r_0 是初始状态, 且 r_n 是接受状态。因为状态只有 p 个, 而序列有 $n+1 > p$ 个状态, 根据鸽巢原理, 必有两个状态相同。设 $r_j = r_l$ 是第一次出现重复的状态, 且 $0 \leq j < l \leq p$ 。那么, 取 $x = s_1 \cdots s_j$, $y = s_{j+1} \cdots s_l$, $z = s_{l+1} \cdots s_n$, 则 $|y| > 0$, $|xy| \leq p$ 。由于 $r_j = r_l$, 所以从 r_j 读入 y 会回到 r_j , 因此对任意 $i \geq 0$, xy^iz 都会到达同一个接受状态 r_n , 即 $xy^iz \in L$ 。

4.2 泵引理的应用

使用泵引理证明语言 L 不是正则的一般步骤：

1. 假设 L 是正则的，则存在泵长度 p 。
2. 选择一个合适的字符串 $s \in L$ ，且 $|s| \geq p$ 。选择 s 时需要技巧，通常要使泵引理的条件能导致矛盾。
3. 对于所有满足泵引理条件的 s 的划分 $s = xyz$ （即 $|xy| \leq p$ ， $|y| > 0$ ），都要能导出矛盾，即 xy^iz 不在 L 之中。因此，需要根据 p 来构造 s ，使得无论 y 怎么取（只要满足条件），通过泵入（例如取 $i = 0$ 或 $i = 2$ 等）都会得到一个不在 L 中的字符串。
4. 因此假设错误， L 不是正则的。

常见的非正则语言例子：

- $\{0^n 1^n : n \geq 0\}$;
- $\{ww : w \in \{0, 1\}^*\}$;
- $\{0^p : p \text{ 是素数}\}$ 。

4.3 从 NFA 到正则表达式的转换

另一种重要的技术是将有限自动机转换为等价的正则表达式。这可以通过状态消除法实现，也可以使用广义非确定性有限自动机（GNFA）。

GNFA（广义非确定性有限自动机）：GNFA 是一种特殊的 NFA，其转移边上的标签可以是任意正则表达式，而不仅仅是单个符号或 ε 。GNFA 有一个唯一的开始状态（没有入边）和一个唯一的接受状态（没有出边）。其他状态之间都有边（如果没有实际转移，则标签为 $\emptyset$ ）。

转换步骤：

1. 给定一个 NFA，首先添加一个新的开始状态（用 ε 边连接到原开始状态）和一个新的接受状态（从所有原接受状态用 ε 边连接到此状态）。这样确保开始状态没有入边，接受状态没有出边。
2. 逐步消除中间状态。每次消除一个状态 q ，对于每一对状态 q_i （指向 q ）和 q_j （从 q 指出），我们需要更新从 q_i 到 q_j 的边上的正则表达式，以反映经过 q 的所有路径。具体地，假设从 q_i 到 q 的边标签为 R_1 ，从 q 到自身的自环标签为 R_2 （如果没有自环，则 $R_2 = \emptyset$ ），从 q 到 q_j 的边标签为 R_3 。那么消除状态 q 后，从 q_i 到 q_j 的路径新增了通过 q 的路径，其正则表达式为 $R_1 R_2^* R_3$ 。将这个表达式与原来从 q_i 到 q_j 的边标签（设为 R_4 ）合并，得到新的标签为 $R_4 \cup R_1 R_2^* R_3$ 。
3. 重复消除中间状态，直到只剩下开始状态和接受状态。此时，从开始状态到接受状态的边上的正则表达式就是所求。

注意：在消除状态时，需要更新所有受影响的边。可能还需要处理自环。

状态消除法：与 GNFA 方法类似，但更直观。直接在原 NFA 上操作，每次选择一个状态消除，并重新连接其他状态。

二、作业题目详细讲解

题目 1

题目

设 $L = \{0^p : p \text{ 是素数}\}$ 。证明 L 不是正则的。

讲解

我们使用泵引理。假设 L 是正则的，那么存在泵长度 p (正整数)。我们要选择一个合适的字符串 $s \in L$ ，且 $|s| \geq p$ ，然后说明对于任何满足泵引理条件的划分，都能导出矛盾。

注意：泵引理是说存在一个 p ，使得对于任意长度 $\geq p$ 的字符串 s ，存在一种划分 $s = xyz$ 满足条件。所以当我们假设 L 正则时，我们并不知道 p 具体是多少，但我们可以根据 p 来构造 s 。通常我们选择 $s = 0^q$ ，其中 q 是一个大于等于 p 的素数。这样的素数总是存在的（比如取大于 p 的某个素数）。

设 $s = 0^q$ ，其中 $q \geq p$ 且 q 是素数。根据泵引理， s 可以写成 $s = xyz$ ，满足：

1. $|xy| \leq p$;
2. $|y| > 0$;
3. 对于所有 $i \geq 0$ ， $xy^iz \in L$ 。

由于 $|xy| \leq p$ 且 s 全部由 0 组成，所以 y 也全部由 0 组成，设 $|y| = k$ ，则 $1 \leq k \leq p$ 。那么 xy^iz 就是 $0^{q+(i-1)k}$ ，因为 x 和 z 也是 0。

根据条件 3，对于任意 $i \geq 0$ ， $xy^iz \in L$ ，即 $0^{q+(i-1)k}$ 的指数必须是素数。特别地，取 $i = q + 1$ ，则 $xy^{q+1}z = 0^{q+qk} = 0^{q(1+k)}$ 。因为 $k \geq 1$ ，所以 $1 + k \geq 2$ ，且 $q \geq 2$ ，因此 $q(1+k)$ 是两个大于 1 的整数的乘积，是合数。所以 $0^{q(1+k)}$ 不在 L 中，矛盾。因此假设错误， L 不是正则的。

注意：这里我们选择 $i = q + 1$ 是为了让指数变得明显是合数。实际上也可以选择其他 i ，但需要确保指数不是素数。例如，取 $i = 0$ ，得到 0^{q-k} ，但 $q - k$ 可能是素数也可能不是，所以不一定矛盾。因此选择 i 使得指数明显是合数很重要。

题目 2

题目

设 $L = \{www : w \in \{0,1\}^*\}$ 。证明 L 不是正则的。

讲解

同样使用泵引理。假设 L 是正则的，存在泵长度 p 。

我们选择字符串 $s = 0^p 1^p 0^p 1^p 0^p 1^p$ 。这个字符串可以看成三个相同的部分 www ，其中 $w = 0^p 1^p$ 。显然 $s \in L$ ，且 $|s| = 6p \geq p$ 。

根据泵引理， s 可以写成 $s = xyz$ ，满足 $|xy| \leq p$ ， $|y| > 0$ ，且对于所有 $i \geq 0$ ， $xy^iz \in L$ 。

由于 $|xy| \leq p$ ，且 s 的前 p 个字符都是 0，所以 y 一定完全由 0 组成，设 $|y| = k$ ， $1 \leq k \leq p$ 。

现在考虑 $i = 0$ ，即泵出 y ，得到字符串 $xz = 0^{p-k} 1^p 0^p 1^p 0^p 1^p$ 。如果 $xz \in L$ ，那么它必须能写成三个完全相同的子串连接的形式。观察 xz ，它由三段组成：第一段是 $0^{p-k} 1^p$ ，第二段是 $0^p 1^p$ ，第三段是 $0^p 1^p$ 。由

于 $k > 0$ ，第一段中 0 的个数为 $p - k$ ，小于第二段和第三段中 0 的个数 p ，因此这三个子串不可能完全相同。所以 $xz \notin L$ ，与泵引理矛盾。因此 L 不是正则的。

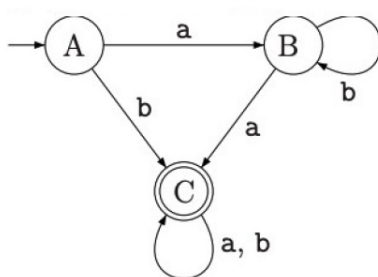
注意：这里选择 $i = 0$ 是因为可以破坏三个部分的一致性。当然也可以选择其他 i ，比如 $i = 2$ ，得到 xy^2z ，那么第一个子串中 0 的个数会增加，但第二个子串不变，同样会破坏一致性。但选择 $i = 0$ 是最简单的。

题目 3

题目

应用课堂上的步骤将给定的 NFA 转换为正则表达式。

补充描述：状态转移图如下：



- 状态 A （初始状态）、 B 、 C （接受状态，双圈）。
- 转移： $A \rightarrow B$ 上有标签 a ； $A \rightarrow C$ 上有标签 b ； $B \rightarrow C$ 上有标签 a ； $B \rightarrow B$ 上有标签 b （自环）； $C \rightarrow C$ 上有标签 a, b （自环）。

讲解

我们使用状态消除法。目标是将 NFA 转换为等价的正则表达式。

步骤 1：将 NFA 转化为 GNFA 形式

我们添加新的开始状态 S 和新的接受状态 F ，并添加 ε 转移：

- 从 S 到 A ： ε
- 从 C 到 F ： ε （因为 C 是原接受状态）

现在，我们有一个 GNFA，状态集合为 $\{S, A, B, C, F\}$ 。

步骤 2：消除状态 A

状态 A 有入边： $S \rightarrow A(\varepsilon)$ ；出边： $A \rightarrow B(a)$ ， $A \rightarrow C(b)$ 。

消除 A 时，我们需要考虑经过 A 的路径：

- $S \rightarrow A \rightarrow B$ ： $\varepsilon a = a$
- $S \rightarrow A \rightarrow C$ ： $\varepsilon b = b$

将这两条路径加到已有的边上（ S 到 B 原来没有边，所以直接添加； S 到 C 原来没有边，直接添加）。所以：

- 添加 $S \rightarrow B$ ： a
- 添加 $S \rightarrow C$ ： b

同时， S 到其他状态的边不变。然后移除状态 A 。

此时 GNFA 状态为 $\{S, B, C, F\}$ ，转移如下：

- $S \rightarrow B: a$
- $S \rightarrow C: b$
- $B \rightarrow B: b$ (自环)
- $B \rightarrow C: a$
- $C \rightarrow C: a \cup b$ (自环)
- $C \rightarrow F: \varepsilon$

步骤 3：消除状态 B

状态 B 有入边： $S \rightarrow B(a)$ ， $B \rightarrow B(b)$ (自环)；出边： $B \rightarrow C(a)$ 。

消除 B 时，要注意 B 有自环，从 S 到 C 的路径可以通过 B 任意次：

- 从 S 到 C 经过 B ： $S \rightarrow B(a)$ ，然后 B 的自环 b^* ，然后 $B \rightarrow C(a)$ ，所以路径为 ab^*a 。
- 另外，从 S 到 C 原来有直接边 b (来自 $S \rightarrow C$)，所以合并后 $S \rightarrow C$ 的标签更新为 $b \cup ab^*a$ 。

消除 B 后，状态变为 $\{S, C, F\}$ ，转移如下：

- $S \rightarrow C: b \cup ab^*a$
- $C \rightarrow C: a \cup b$ (自环)
- $C \rightarrow F: \varepsilon$

步骤 4：消除状态 C

状态 C 有入边： $S \rightarrow C(b \cup ab^*a)$ ， $C \rightarrow C(a \cup b)$ (自环)；出边： $C \rightarrow F(\varepsilon)$ 。

消除 C 时，从 S 到 F 的路径： $S \rightarrow C(b \cup ab^*a)$ ，然后 C 的自环 $(a \cup b)^*$ ，然后 $C \rightarrow F(\varepsilon)$ 。所以 S 到 F 的路径标签为 $(b \cup ab^*a)(a \cup b)^*$ 。

因此，所求正则表达式为 $(b \cup ab^*a)(a \cup b)^*$ 。

复习小结：第四节课重点掌握泵引理的内容、证明思路和应用，以及如何使用状态消除法（或 GNFA）将 NFA 转换为正则表达式。作业题目分别练习了泵引理证明非正则语言和 NFA 到正则表达式的转换。

第五节课复习资料

一、本节课知识点讲解

5.1 图灵机的定义与基本操作

图灵机是计算理论中最核心的计算模型，它由一个无限长的纸带、一个读写头和一套控制规则（状态转移函数）组成。图灵机提供了算法执行的形式化模型。

形式化定义（七元组） 一个（确定型）图灵机 M 是一个七元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ ，其中：

1. Q : 有限的状态集合。
2. Σ : 输入字母表 (不包含空白符 $\sqcup$ 或 $\sqcup$, 注意这里的 $\sqcup$ 是空白符, 字符串之外的所有格子都默认填充它)。
3. Γ : 纸带字母表, 满足 $\Sigma \subseteq \Gamma$ 且包含空白符 (通常记作 $\sqcup, B$ 或 $_$)。
4. δ : 转移函数, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 。
 - 输入: 当前状态 q 和读写头下方符号 a 。
 - 输出: 新状态 q' 、要写入的符号 b 以及读写头的移动方向 (L 左移或 R 右移)。
5. $q_0 \in Q$: 初始状态。
6. $q_{accept} \in Q$: 接受状态 (唯一, 进入此状态则停机并接受输入)。
7. $q_{reject} \in Q$: 拒绝状态 (唯一, 进入此状态则停机并拒绝输入)。

图灵机如何运行

1. 开始时, 输入字符串写在纸带上, 其余格子为空白符。读写头指向输入最左端的符号。机器处于初始状态 q_0 。
2. 每一步, 机器根据当前状态 q 和读写头下符号 a , 查找转移函数 $\delta(q, a) = (q', b, D)$ 。
 - 将当前格子的符号改写为 b 。
 - 将状态变为 q' 。
 - 根据 D 将读写头向左 (L) 或向右 (R) 移动一格。
3. 当机器进入 q_{accept} 或 q_{reject} 状态时, 立即停机, 并相应输出“接受”或“拒绝”。
4. 如果转移函数 δ 对当前 (q, a) 无定义, 机器也会停机 (通常视为拒绝)。

图灵机的基本算法模式

- 扫描与计数: 通过左右移动读写头来遍历字符串, 用不同的符号进行标记, 以计数或检查模式。
- 记忆与比较: 通过状态或纸带上的特殊标记来“记住”之前读取的符号, 以便与后续符号进行比较 (例如判断回文)。
- 迭代与循环: 通过设计状态间的循环转移, 实现对某一段纸带的重复处理。

时间复杂度 (初步) 对于判定语言 $L \subseteq \{0, 1\}^*$ 的图灵机 M , 我们说 M 在时间 $T(n)$ 内判定 L , 如果对于每个长度为 n 的输入 x :

1. M 在至多 $T(|x|)$ 步内必然停机 (到达 q_{accept} 或 q_{reject})。
2. 若 $x \in L$, 则 M 接受 x 。
3. 若 $x \notin L$, 则 M 拒绝 x 。

注意: 这里的 $L \subseteq \{0, 1\}^*$ 并不是特例, 而是因为计算机科学中任何对象 (整数、图、程序代码、矩阵等) 都可以通过编码映射为一段二进制字符串。此外, 这个概念是后续讨论计算复杂性 (P、NP 类) 的基础。

二、作业题目详细讲解

题目 1

题目

输入是两个二进制数 x 和 y （最高位在左边），由符号 $\#$ 分隔，例如 $100\#100$ 。假设没有前导零，除非数字本身是 0。设计一个图灵机，如果 x 和 y 相等，则接受。

讲解

我们的目标是逐位比较 x 和 y 的每一位。由于输入格式固定为 $x\#y$ ，且没有前导零，我们可以从左到右依次处理。

核心思路：

1. 逐位配对与标记：从 x 的最左端开始，读取一位（0 或 1），将其标记为一个特殊符号（例如 X ），然后向右移动找到 $\#$ ，再继续向右找到 y 的对应位（第一个尚未被标记的位）。检查该位是否与 x 刚才读取的位相同。
2. 匹配处理：如果相同，则将 y 的该位标记为另一个特殊符号（例如 Y ），然后向左移动回到 x 部分，找到下一个未被标记的位，重复此过程。
3. 长度与结束检查：在比较过程中，如果出现位数不匹配（例如 x 还有位但 y 已到末尾，或符号不匹配），则立即拒绝。当 x 的所有位都被标记后，需要检查 y 是否也恰好没有剩余位了。如果是，则接受。

状态设计与转移逻辑：

- **start**: 初始状态。查看第一个符号。
 - 如果是 $\#$ ，说明 x 为空，则直接跳到检查 y 是否也为空的状态 (**check_y_end**)。
 - 如果是 0 或 1，将其改写为 X （标记为已处理），并分别进入状态 **check0** 或 **check1**，开始寻找 y 的对应位。
 - 如果是空白 ($\square$)，说明输入完全为空（ x 和 y 都是空串），直接接受。
- **check0 / check1**: 向右移动，跳过已经处理过的 0、1（理论上这里不会出现 X ），寻找分隔符 $\#$ 。
- **verify0 / verify1**: 在找到 $\#$ 后进入此状态，继续向右跳过 Y （ y 中已匹配的位），寻找 y 的第一个未匹配位。它期望找到一个与 x 对应位相同的符号（**check0** 对应 0，**check1** 对应 1）。如果找到，将其改写为 Y ，然后进入 **go_back** 状态准备下一轮比较；如果找到的是另一个符号或空白，则拒绝。
- **go_back**: 向左移动，目标是回到 x 部分下一个待比较的位。它会跳过 Y 和 $\#$ ，当遇到 X 时，说明回到了 x 区域的起点，此时右移一格（回到 **start** 状态）开始处理下一个 X 右边的位（即 x 的下一位）。如果意外遇到空白，说明逻辑错误，拒绝。
- **check_y_end**: 当 **start** 状态遇到 $\#$ （ x 已处理完）时进入。它向右检查，应该只能看到 Y （ y 中已匹配的位）。当遇到空白时，说明 y 也恰好处处理完，接受。如果遇到 0 或 1，说明 y 比 x 长，拒绝。

答案展示

```
input: '100#100' # 测试
blank: ' '
start state: start
```

```
table:
  start:
    0: {write: X, R: check0}
    1: {write: X, R: check1}
    '#': {R: check_y_end}
    ' ': {R: accept} # 都空则接受
  check0:
    0: R
    1: R
    Y: R # 已标记处理过则可跳过
    '#': {R: verify0}
  check1:
    0: R
    1: R
    Y: R # 已标记处理过则可跳过
    '#': {R: verify1}
  verify0:
    0: {write: Y, L: go_back}
    1: {R: reject}
    ' ': {R: reject}
    Y: R # 继续找0
  verify1:
    1: {write: Y, L: go_back}
    0: {R: reject}
    ' ': {R: reject}
    Y: R # 继续找1
  go_back:
    Y: L
    '#': L
    X: {R: start}
    0: L
    1: L
    ' ': {R: reject}
  check_y_end:
    Y: R # 继续检查Y
    ' ': {R: accept} # Y结束, 匹配
    0: {R: reject} # 超出, 拒绝
    1: {R: reject} #
  accept:
  reject:
```

题目 2

题目

设语言 $L = \{0^n : n \text{ 是斐波那契数}\}$ 。设计一个判定 L 的图灵机。

讲解

斐波那契数列定义为: $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, \dots$, 即 $F_n = F_{n-1} + F_{n-2}$ ($n \geq 2$)。L 中包含 $0^{F_2}(0), 0^{F_3}(00), 0^{F_4}(000), 0^{F_5}(00000), \dots$ 等字符串。我们需要判断输入的 0 串长度是否在这个数列中。

核心思路 (模拟斐波那契数列生成与比较): 我们不能直接计算 n 是否为斐波那契数, 因为图灵机无法进行复杂的算术运算。但我们可以用动态标记的方法来模拟:

1. 初始化两个“指针”: 在纸带上用两个特殊标记 (比如 **x**) 来代表当前考虑的两个连续的斐波那契数 F_k 和 F_{k+1} 在输入串中的“结束位置”。初始时, 第一个 **x** 标记第一个 0, 第二个 **x** 标记第二个 0。注意, 这里我们把输入串的起始位置与斐波那契数的计数对应起来。
2. 迭代检查与移动:
 - 比较当前输入串的长度是否等于第二个 **x** 的位置 (即当前考虑的 F_{k+1})。如果输入恰好结束在第二个 **x** 处, 当然就要接受。
 - 如果输入串的长度超过了第二个 **x**, 说明 $n > F_{k+1}$, 那么我们需要计算下一个斐波那契数 $F_{k+2} = F_k + F_{k+1}$, 并更新两个“指针”。
 - 如何计算下一个位置: 将第一个 **x** 向右移动到第二个 **x** 的当前位置, 这代表了旧的 F_{k+1} 变成了新的 F_k 。(同时要设法删掉旧的第一个 **x**) 然后, 从新的第一个 **x** (即旧的第二个 **x**) 位置出发, 向右“数”出与旧的 F_k (即旧的第一个 **x** 到纸带起点的距离) 相等的 0 的个数, 并在那里放置新的第二个 **x**, 这代表了新的 F_{k+1} 。这个“数”的过程通过状态转移和来回移动实现。重复此过程。
3. 边界与拒绝:
 - 如果输入串长度在两个连续的斐波那契数位置之间 (即超过第一个 **x** 但未达到第二个 **x** 就遇到空白), 则拒绝。
 - 如果输入是空串 ($n = 0$), 根据定义 0 不是斐波那契数, 不过以题目的具体要求为准即可。

状态设计与转移逻辑概要:

- **start**: 初始化。标记前两个 0 为 **x**, 建立初始的“指针”。
- **check_match**: 位于第二个 **x** (F_{k+1} 的边界)。检查前方是否是空白。若是, 说明输入长度正好是斐波那契数, 接受。若不是, 则标记下一个 0 为 **x** (这并不是说下一个位置也是斐波那契数, 而是为了做标记, 以协助继续计算与查找), 并进入状态 **find_the_first_x**, 开始计算下一个斐波那契数的位置。
- **check_excess**: 位于非斐波那契的边界。检查前方是否是空白。若是, 说明输入长度不是斐波那契数, 拒绝。若不是, 同样标记并进入计算流程。
- **find_the_first_x, find_the_second_x, judge, suspect1, suspect2, trust1, trust2**: 这些状态共同协作, 完成复杂的“移动第一个 **x**”和“测量距离以放置新的第二个 **x**”的任务。它们通过左右来回移动、临时改变标记、判断是否到达正确位置等操作, 模拟斐波那契数列的递推过程。其中, 代码中的 **trust** 和 **suspect** 路径分别对应了输入长度正好等于和大于当前斐波那契数边界两种情况的不同处理逻辑。具体理解可见代码注释。

答案展示

```
input: '0000000000000' # 以13为例测试
blank: ' '
```

```

start state: start
table:
  start:
    ' ': {R: accept} # 空输入, n=0, 接受
    0: {write: X, R: init_second} # 标记第一个0, 初始化第二个斐波那契数
  init_second:
    ' ': {R: accept} # 只有一个0, n=1, 接受
    0: {write: X, R: check_match} # 标记第二个0, 进入检查匹配状态 (因为2本身是斐波那契数, 应当进入match而非excess)

# 当前是一个斐波那契的位置
  check_match:
    ' ': {R: accept} # 无剩余0, 匹配斐波那契数, 接受
    0: {write: X, L: find_the_first_x} # 说明数更长, 还要继续往右读 (这里写X并不是因为是斐波那契数, 而是为了标记现在)
# 当前是一个非斐波那契的位置
  check_excess:
    ' ': {R: reject} # 超出输入长度, 拒绝
    0: {write: X, L: find_the_first_x} # 说明数更长, 还要继续往右读

# 第一个x表示当前存着的最大的斐波那契数
  find_the_first_x:
    0: L # 向左移动跳过0
    X: {L: find_the_second_x} # 找第二个x
# 第二个x表示当前最左x标识符, 实际代表右边是否达到斐波那契数
  find_the_second_x:
    0: L # 继续左移
    X: {write: 0, L: judge} # 将X恢复为0, 准备右移

# 关键区分
# trust向右行进目的是进行判断是否accept
# suspect向右行进目的是进行判断是否reject
  judge:
    ' ': {R: trust1} # 到达最左的同时, 说明右边也已经右移到了斐波那契的位置
    0: {write: X, R: suspect1} # 还没有到达最左, 说明右边还没达到斐波那契的位置
  suspect1:
    0: R # 向右移动跳过0
    X: {R: suspect2} # 遇到X, 进入右移状态2
  suspect2:
    0: R # 继续右移
    X: {write: 0, R: check_excess} # 将X恢复为0, 检查是否超出
  trust1:
    0: R # 向右移动跳过0
    X: {R: trust2} # 遇到X, 进入右移状态2
  trust2:
    0: R # 继续右移
    X: {R: check_match} # 遇到X, 返回检查匹配状态
accept:
reject:

```

题目 3

题目

在图灵机的定义中，读写头可以向左或向右移动。允许读写头向左、向右或保持不动的图灵机称为允许静止移动的图灵机，形式化定义为： $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ 。证明如果语言 L 可以被允许静止移动的图灵机判定，那么 L 也可以被标准图灵机判定。

讲解

我们需要证明，允许静止移动 (S) 这个功能并没有增强图灵机的计算能力。任何用这种增强版图灵机能判定的语言，用原来的标准图灵机（只允许 L 和 R ）也能判定。证明方法是构造性的：给定一个允许静止移动的图灵机 M_S ，我们显式地构造一个标准图灵机 M 来模拟 M_S 的行为，使得两者判定相同的语言。

证明思路：关键在于如何用标准的左右移动 (L, R) 来模拟静止移动 (S)。一个直观的想法是：当 M_S 要执行 S 动作时，让 M 先向右移动一格，然后再向左移动一格，这样读写头就回到了原位。同时， M 需要在状态中记住它正在模拟一个 S 动作，以控制这两步移动。

形式化构造：设允许静止移动的图灵机 $M_S = (Q, \Sigma, \Gamma, \delta_S, q_0, q_{accept}, q_{reject})$ ，其中 $\delta_S : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ 。

我们构造标准图灵机 $M = (Q', \Sigma, \Gamma, \delta, q'_0, q_{accept}, q_{reject})$ ，其中 q_{accept} 和 q_{reject} 与 M_S 相同。

1. 状态集： $Q' = Q \cup \{q_q^{stay} \mid q \in Q\}$ ，为每个 $q \in Q$ 加一个辅助状态 q_q^{stay} （用来完成“右移再左移”）。
2. 初始状态： $q'_0 = q_0$ 。
3. 转移函数：
 - 若 $\delta_S(q, a) = (p, b, D)$ 且 $D \in \{L, R\}$ ，则 $\delta(q, a) = (p, b, D)$ （直接复制）。
 - 若 $\delta_S(q, a) = (p, b, S)$ ，用两步模拟停留：
 - 1) $\delta(q, a) = (q_q^{stay}, b, R)$;
 - 2) 对任意 $c \in \Gamma$ ， $\delta(q_q^{stay}, c) = (p, c, L)$ 。

如此，先右移一格再左移回原格，进入同样的下一状态 p ，头位置与 M_S 一致。

正确性：通过上述构造， M 严格地模拟了 M_S 的每一步计算。对于 L 和 R 移动， M 直接复制。对于 S 移动， M 通过一系列状态和移动，最终效果也是在原位置写入了相同的符号，并且读写头停留在原位置（经过校正后），然后进入相同的下一个状态。因此，对于任何输入串， M 和 M_S 的接受/拒绝行为完全一致。由于 M 是标准图灵机，这就证明了标准图灵机的计算能力至少与允许静止移动的图灵机一样强。

复习小结：第五节课重点掌握图灵机的形式化定义（七元组）和基本运行原理，理解其作为算法形式化模型的核心地位。通过作业，练习了图灵机设计的基本方法：逐位比较、利用标记模拟递推过程。同时，通过证明题理解了图灵机模型某些定义上的变种（如允许静止移动）并不改变其根本的计算能力，证明的关键在于如何用原始操作模拟新增的操作。

第六节课复习资料

一、本节课知识点讲解

6.1 识别 (Recognizing) 与判定 (Deciding) 的区别

在图灵机的范畴内，我们需要区分两个重要概念：

- 识别 (**Recognizing**): 如果存在一个图灵机 M ，使得对于任意输入 x ，当 x 属于语言 L 时 M 最终会接受 x ，而当 x 不属于 L 时 M 要么拒绝要么永远不停机（即可能进入无限循环），那么我们说 M 识别 (**recognizes**) 了语言 L ，并且称 L 是图灵可识别的 (**Turing recognizable**)。
- 判定 (**Deciding**): 如果存在一个图灵机 M ，使得对于任意输入 x ， M 总是能在有限步内停机，并且当 $x \in L$ 时接受， $x \notin L$ 时拒绝，那么我们说 M 判定 (**decides**) 了语言 L ，并且称 L 是可判定的 (**decidable**)。

显然，每个可判定的语言都是图灵可识别的，但反过来不一定成立（例如，著名的停机问题对应的语言就是图灵可识别但不可判定的）。

6.2 可计算函数 (Computable Function)

设 $f: \{0,1\}^* \rightarrow \{0,1\}^* \cup \{\perp\}$ 是一个部分函数（即某些输入上可能没有定义，用 $\perp$ 表示）。我们称一个图灵机 M 在时间 $T(n)$ 内计算 (**computes**) f ，如果对于每个输入 x （且 $f(x) \neq \perp$ ）， M 在至多 $T(|x|)$ 步内停机，并且此时纸带上除了空白符外剩下的内容正好是 $f(x)$ 。

6.3 图灵机的变体及其等价性 (重点)

为什么图灵机是一个好的计算模型定义？一个重要原因是它的稳健性 (**robustness**)：许多看起来增强或修改了图灵机定义得到的变体，在计算能力上并没有超过标准图灵机（即它们能判定的语言类是一样的），而且这些变体之间的运行时间也只差一个多项式因子。这使得图灵机模型具有很强的通用性。

变体 1: 字母表改变 结论：如果语言 L 能被一个字母表为 Γ 的图灵机在时间 $T(n)$ 内判定，那么它也能被一个字母表为 $\{0,1,\text{blank}\}$ 的图灵机在时间 $O(\log |\Gamma| \cdot T(n))$ 内判定。

证明思路：将 Γ 中的每个符号用 $k = \lceil \log |\Gamma| \rceil$ 个二进制位进行编码。模拟原图灵机的一步时，需要读取 k 位来解码原符号，然后根据转移函数确定要写入的新符号（也是 k 位），再移动读写头。因此，一步模拟需要 $O(k)$ 步，总时间就是 $O(k \cdot T(n))$ 。

变体 2: 单向图灵机 (One-way Turing Machine) 这种图灵机的纸带有一个左边界（不能向左越过），并且规定在左边界上只能向右移动。

结论：单向图灵机与标准图灵机计算能力等价，且时间开销只差常数倍。

证明思路：将标准图灵机的双向无限纸带“折叠”到单向纸带上。具体地，将标准纸带上的位置 $0, 1, 2, \dots$ 映射到单向纸带的位置 $0, 2, 4, \dots$ ，而将位置 $-1, -2, -3, \dots$ 映射到单向纸带的位置 $1, 3, 5, \dots$ 。这样，模拟一步标准移动时，单向图灵机需要移动两格（因为相邻的两个逻辑位置在物理位置上可能相隔 2）。同时，单

向图灵机需要用状态记住当前读写头是在“正半轴”还是“负半轴”，以决定移动方向。因此，一步模拟需要 $O(1)$ 步。

变体 3: 多带图灵机 (Multi-tape Turing Machine) 一个 k 带图灵机有 k 条独立的纸带，每条纸带都有自己的读写头，转移函数同时基于所有读写头下的符号，并决定每个头写入的符号和移动方向。

结论：任何 k 带图灵机在时间 $T(n)$ 内判定的语言，也能被一个标准单带图灵机在时间 $O(T(n)^2)$ 内判定。

证明思路：这是本节课的重点，也是作业题的核心思想。我们用一条纸带模拟 k 条纸带。具体方法是：

- 将 k 条纸带的内容交错存储：即单带上的位置依次存储第 1 条纸带的第 1 个符号、第 2 条纸带的第 1 个符号、……、第 k 条纸带的第 1 个符号，然后是第 1 条纸带的第 2 个符号、第 2 条纸带的第 2 个符号……以此类推。
- 为了标记每个纸带的读写头位置，我们为字母表中的每个符号 a 引入一个带“帽”的版本 $\hat{a}$ ，表示该位置对应纸带的读写头正指向这个符号。
- 模拟一步多带图灵机的操作时，标准图灵机需要扫描整个已使用的纸带区域，收集所有 k 个读写头下的符号（通过寻找带帽的符号），然后根据原转移函数计算出新的状态、要写入的符号以及每个头的移动方向，然后它就可以再次扫描纸带来更新这些带帽符号（移动“帽”的位置）了。由于 k 是常数，一次扫描需要 $O(T(n))$ 时间。因此，模拟一步需要 $O(T(n))$ 时间，总共 $T(n)$ 步，总时间就是 $O(T(n)^2)$ 。

注意：平方时间是较松的界限，实际可优化到 $O(T(n) \log T(n))$ ，但课程中通常只要求掌握平方模拟。

变体 4: RAM 图灵机 (Random Access Memory Turing Machine) 这种图灵机除了常规纸带外，还有一个通过地址随机访问的“内存”（一个无限数组）。它有一条特殊的地址带，用于指定要读写内存的哪个位置。

结论：如果语言 L 能被一个 RAM 图灵机在时间 $T(n)$ 内判定，那么它也能被一个多带图灵机在时间 $O(T(n)^3)$ 内判定（如果地址长度有界，则只需 $O(T(n)^2)$ 时间）。

证明思路：用一条额外的纸带来模拟内存数组，存储 (地址, 值) 对。当需要访问某个地址 i 时，模拟器扫描这条纸带寻找地址为 i 的项。每次 RAM 要读写某个地址时，图灵机的读写头必须在带子上线性扫描来寻找这个地址。由于带子上存储的记录总数最多可达 $T(n)$ ，每次寻道在最坏情况下需要移动 $O(T(n))$ 步。此外，在单位代价模型下，RAM 的一个单元可以存放非常大的数，且对其进行算术运算只需 1 步。但在 $T(n)$ 步中，通过不断的加法或位移，RAM 寄存器中的数值长度（位数）最高可以增长到 $O(T(n))$ 位。因此标准图灵机模拟时又增加了一重 $O(T(n))$ 。总共 $(T(n))$ 次访问，总时间就是 $O(T(n)^3)$ 。

6.4 复杂性类 P 的定义

基于多带图灵机，我们可以定义时间复杂度类：

设 $T: \mathbb{N} \rightarrow \mathbb{N}$ 是一个函数。语言 L 属于 $\mathbf{DTIME}(T(n))$ 当且仅当存在一个常数 c 和一个多带图灵机 M ，使得 M 在时间 $c \cdot T(n)$ 内判定 L 。

多项式时间类 P 定义为所有多项式时间可判定的语言的集合：

$$P = \bigcup_{c \geq 1} \mathbf{DTIME}(n^c)$$

也就是说，一个语言 L 属于 P ，当且仅当存在一个多带图灵机和一个常数 k ，使得该图灵机能在 $O(n^k)$ 时间内判定 L 。

重要备注：由于上述各种图灵机变体之间模拟只产生多项式时间开销，所以 P 的定义不依赖于具体使用哪种图灵机模型（标准单带、多带、RAM 等）。这是 P 类作为“易解问题”类的一个稳健性保证。

6.5 图灵机的可数性

集合的可数（**countable**）是指其元素可以与自然数一一对应（即存在一个从自然数到该集合的单射）。有限集和可数无限集都是可数的。

结论：所有图灵机（包括各种变体）的集合是可数的。

证明思路：每个图灵机都可以用一个有限的字符串（例如它的形式化描述）来编码。因为有限字母表上的所有有限字符串的集合是可数的（可以按长度和字典序枚举），所以图灵机的集合作为其子集也是可数的。这个结论非常重要，它意味着存在“不可计算”的函数（因为所有函数的集合是不可数的），从而引出计算理论中的许多深刻结果。

二、作业题目详细讲解

题目 1

题目

定义一个二维图灵机为每个磁带都是无限网格的图灵机（机器不仅可以左右移动，还可以上下移动）。证明如果语言 L 可以被二维图灵机在时间 $T(n)$ 内判定，那么 $L \in \text{DTIME}(T(n)^2)$ 。

讲解

我们只需给出直接的模拟方案：在 $T(n)$ 步内，二维头最多触及 $O(T(n))$ 行和 $O(T(n))$ 列，共 $O(T(n)^2)$ 个格子；单带模拟每步扫描这一活动区即可，单步成本 $O(T(n)^2)$ ：用分块/行分隔让一次扫描成本 $O(T(n))$ ，于是总时间 $O(T(n)^2)$ 。下面给出简明构造。

构造：

1. 布局：在一维纸带上按行序存储有限的“活动矩形”：先存第 1 行的已访问列，紧跟一个行分隔符，再存第 2 行，依此类推；每个单元记录其符号，并用“帽符号”标记当前二维头所在单元。
2. 头位置记录：帽符号唯一地出现在某个单元上，表示当前二维坐标；无需显式存 (x, y) ，只靠帽子定位。
3. 模拟一步：
 - 从纸带起点扫一遍，找到帽符号，得到当前符号。
 - 依据二维机的转移，写新符号、决定方向；在扫的过程中把帽子移到目标单元（如果目标在同一行则在当前行中左/右移，否则越过分隔符到上一行或下一行；理论上此时必能到达目标的行/列，否则说明此位置尚未被构造，则在相应位置插入空白单元并放置帽子）。
 - 该扫描长度与活动矩形的周长线性相关，为 $O(T(n))$ 。
4. 时间界：总步数 $T(n)$ ，每步 $O(T(n))$ 扫描/更新，故总 $O(T(n)^2)$ 。

结论：因此，任何被二维图灵机在 $T(n)$ 时间内判定的语言 L ，都存在一个标准图灵机在 $O(T(n)^2)$ 时间内判定它，即 $L \in \text{DTIME}(T(n)^2)$ 。

题目 2

题目

定义一个单带双头图灵机为只有一个磁带但有两个独立读写头的图灵机。每个头可以同时独立地读、写和移动。两个头可以同时读取同一个磁带单元。冲突解决如下：

- 如果两个头试图同时写入同一个单元，第一个头先写，第二个头覆盖之前的内容。
- 如果第一个头试图写而第二个头试图读同一单元，第一个头先写，第二个头读取更新后的内容。
- 如果第一个头试图读而第二个头试图写同一单元，第一个头先读，读取旧内容，然后第二个头更新内容。

证明如果语言 L 可以被单带双头图灵机在时间 $T(n)$ 内判定，那么 $L \in \text{DTIME}(T(n)^2)$ 。

讲解

核心：用单头 TM 的三轨纸带模拟双头单带机。（三轨和标准是等价的）

构造要点：

1. 三轨布局：轨道 1 存内容；轨道 2 用 H_1 标记头 1 位置；轨道 3 用 H_2 标记头 2 位置。帽子唯一。
2. 模拟一步：
 - 先左到右扫描一次，找到 H_1, H_2 ，读出对应符号 a_1, a_2 ，在状态中记下，按双头机转移算出 (q', b_1, b_2, D_1, D_2) 。
 - 再扫一遍按规则写与移：先执行头 1 的写/移，再执行头 2 的读/写/移，若同格写则头 2 覆盖（这样就可以自然地实现冲突规则）。移动帽子到新格；若需扩展纸带则插空白。
3. 时间：已用长度 $O(T(n))$ ，每步两次扫描 $O(T(n))$ ，共 $T(n)$ 步，时间 $O(T(n)^2)$ 。

结论：因此， L 可以被标准图灵机在 $O(T(n)^2)$ 时间内判定，即 $L \in \text{DTIME}(T(n)^2)$ 。

题目 3

题目

证明多带图灵机的数量是可数的。

讲解

我们需要证明所有多带图灵机（包括不同带数、不同状态数、不同转移函数等等）构成的集合是一个可数集。可数集是指其元素可以与自然数建立一一对应（或者至少存在一个从自然数到该集合的满射，即可以列出来）。

证明思路：

每个图灵机都可以用一个有限的字符串来完全描述（因为它的组件：状态集、字母表、转移函数等都是有限

的)。而所有有限字符串的集合是可数的。因此，图灵机的集合是可数集的一个子集，从而也是可数的。

具体证明：

1. 图灵机的有限描述：

一个 k 带图灵机 (k 是正整数) 由七部分组成 (上述已述)。

所有这些组件都是有限的 (除了 k 本身, 但 k 是正整数, 也是可数的)。因此, 我们可以将整个图灵机编码成一个字符串。

2. 编码为字符串：

我们可以设计一种编码方案, 将上述组件表示为一个二进制字符串。例如：

- 用二进制数字表示 k 。
- 将状态和符号用二进制串编号。
- 转移函数可以用一个列表表示, 列表中的每一项是 $(q, a_1, \dots, a_k, q', b_1, \dots, b_k, D_1, \dots, D_k)$ 的形式。
- 整个描述可以拼接起来, 并用特殊分隔符隔开。

重要的是, 每个图灵机都有多种编码方式, 但至少存在一种编码。我们固定一种编码方案。

3. 可数性论证：

- 所有有限二进制字符串的集合 $\{0, 1\}^*$ 是可数的。这是因为我们可以按长度和字典序枚举它们：先枚举长度为 0 的字符串 (空串), 然后长度为 1 的字符串 (0,1), 长度为 2 的字符串 (00,01,10,11), 等等。
- 每个多带图灵机都对应至少一个二进制字符串 (它的编码), 而不同的图灵机一定有不同编码。因此, 所有多带图灵机的集合与一个二进制字符串集合的子集一一对应。由于可数集的子集也是可数的, 所以多带图灵机的集合是可数的。

4. 处理不同的带数：

注意, 多带图灵机包括 $k = 1, 2, 3, \dots$ 所有可能。每个固定的 k , k 带图灵机的集合是可数的 (如上所述)。可数多个可数集的并集仍然是可数的 (因为我们可以用“对角线法”枚举所有并集中的元素)。因此, 所有多带图灵机的集合是可数的。

结论：多带图灵机的数量是可数的。

复习小结：第六节课重点掌握图灵机各种变体 (二维、双头、多带、RAM 等) 与标准图灵机的等价性, 特别是如何用标准图灵机模拟这些变体并分析时间开销 (通常是平方或多项式时间)。同时, 理解图灵机的可数性证明, 这为后续讨论不可计算性问题 (如停机问题) 奠定了基础。作业题目要求我们具体构造模拟并分析时间, 需要熟练掌握编码和模拟的基本技巧。